

Module 3: AI Tools

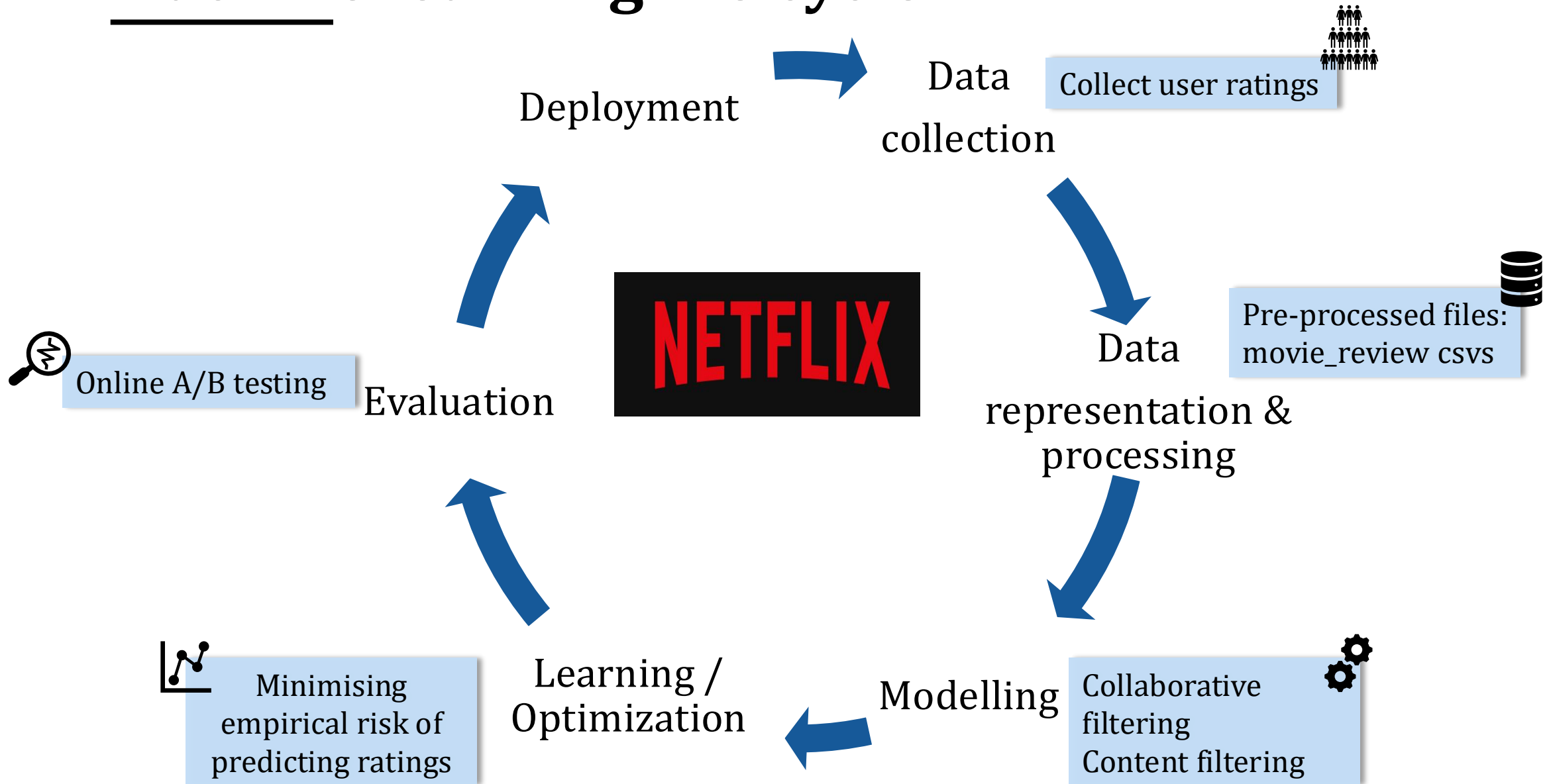
Introductory lecture

Filip Kronström

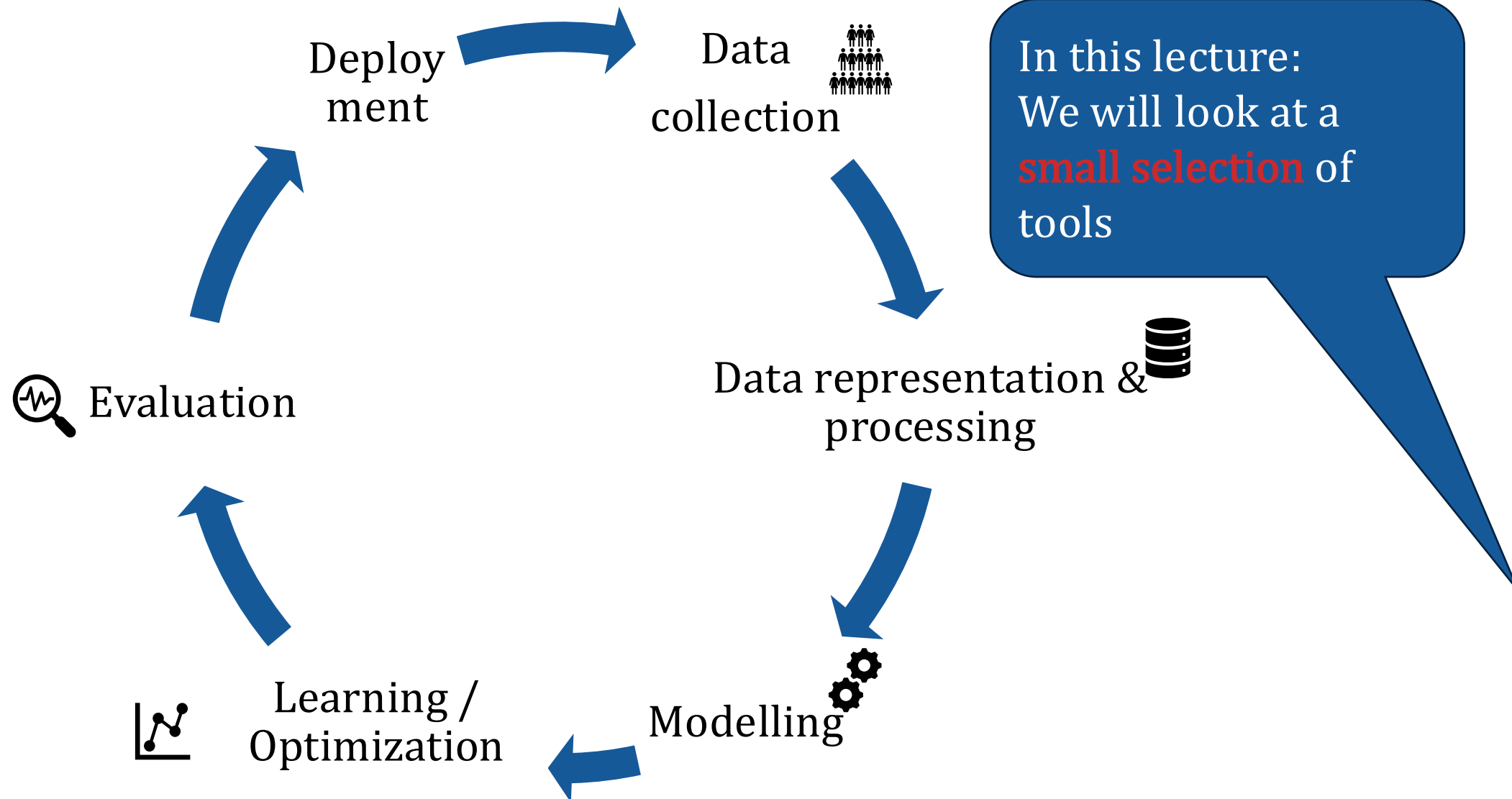
`filipkro@chalmers.se`

February 3, 2026

Machine learning life cycle



AI tools in ML life cycle



What is an AI tool?

- **Here:** something that helps you implement AI systems
- Specifics depend on **nature** of the system

Statistical learning

- Model representation
- Data collection
- Optimization
- Imputation
- Probabilistic programming
- ...

Symbolic/knowledge-based

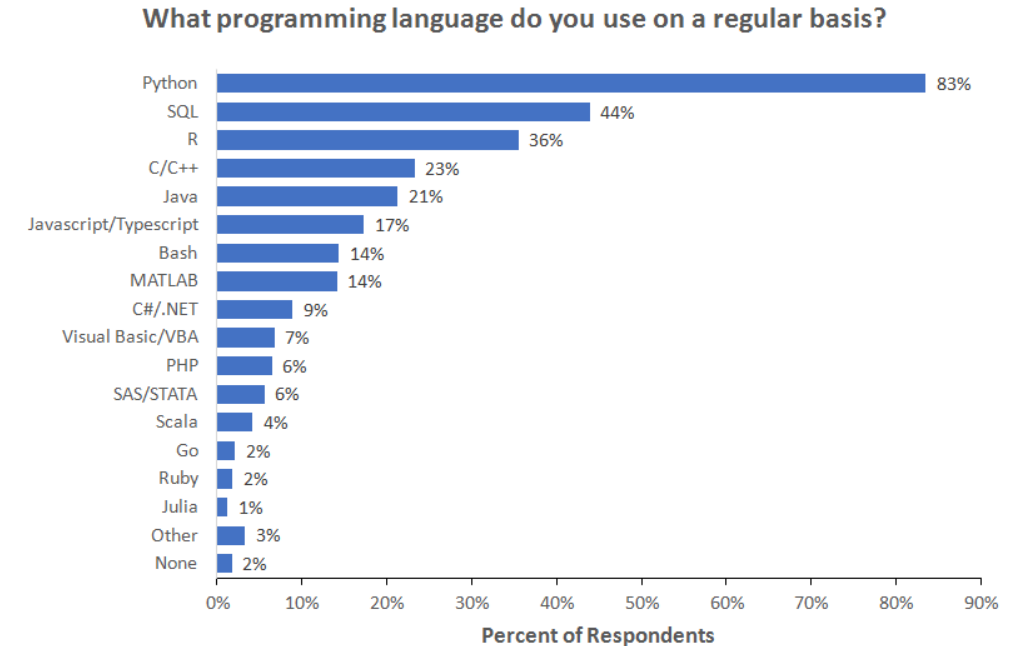
- Knowledge representation
- Optimization
- ...

Limitations

1. We will focus on (statistical) **machine learning**
 - Tool selection is biased towards such systems
2. Most of this lecture is relevant for prototyping
 - Additional steps necessary for taking systems to production

Programming languages

- We will refer to **Python** modules
- Used by largest ML frameworks
 - scikit-learn
 - PyTorch
 - TensorFlow, Keras



Note: Data are from the 2018 Kaggle Machine Learning and Data Science Survey. You can learn more about the study here: <http://www.kaggle.com/kaggle/kaggle-survey-2018>. A total of 18827 respondents answered the question.

1. **Data** representation & processing

What is “typical” ML data?

- When we learn ML, data is stored in **matrices/vectors**
- For example, a classical table of features X and labels Y

$X =$

-2	1	3	4
3	4	5	12
2	5	1	2
1	1	2	4

Features

Observations

$Y =$

-1
1
-1
1

Labels

More general data

- More general data do not have natural tabular representations
- For example, **time series** of medical patients may have
 - Different time-scale
 - Different length
 - Different possible events

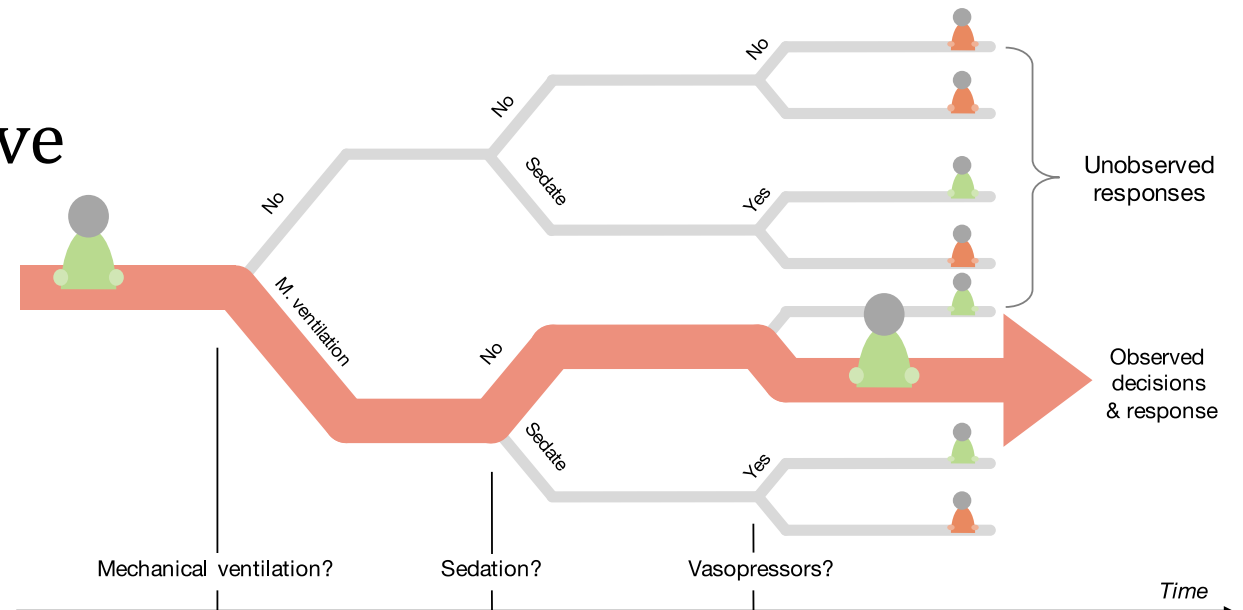
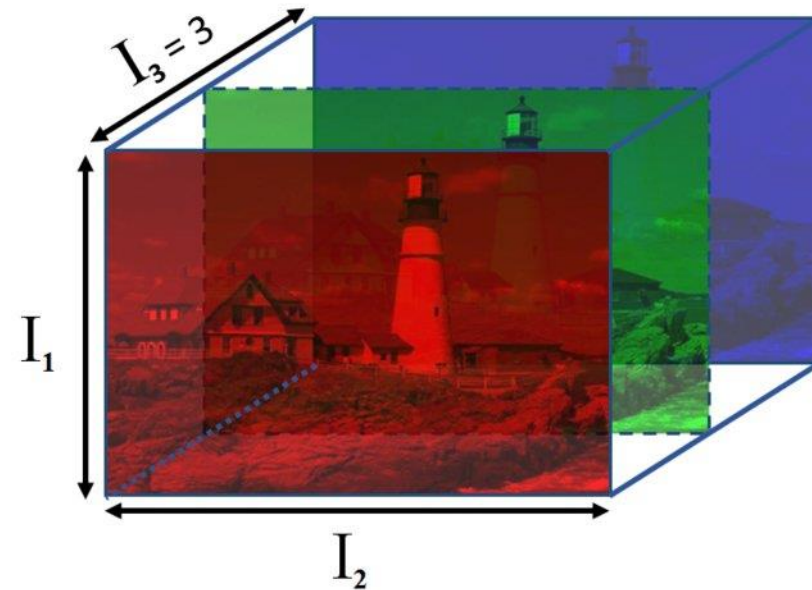


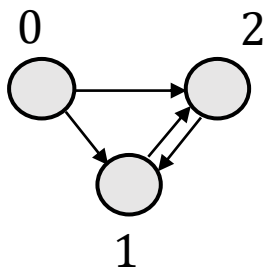
Image data

- (Color) Image data is not easily stored in 2D matrices. Instead, **tensors**!
- 3D “matrices” where each depth dimension represents a different color



Graphs

- Graphs are used to represent relations of people, atoms, cities, ...
- Can be represented in many ways



Adjacency matrix

$$G = \begin{bmatrix} 0 & 1 & 1 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

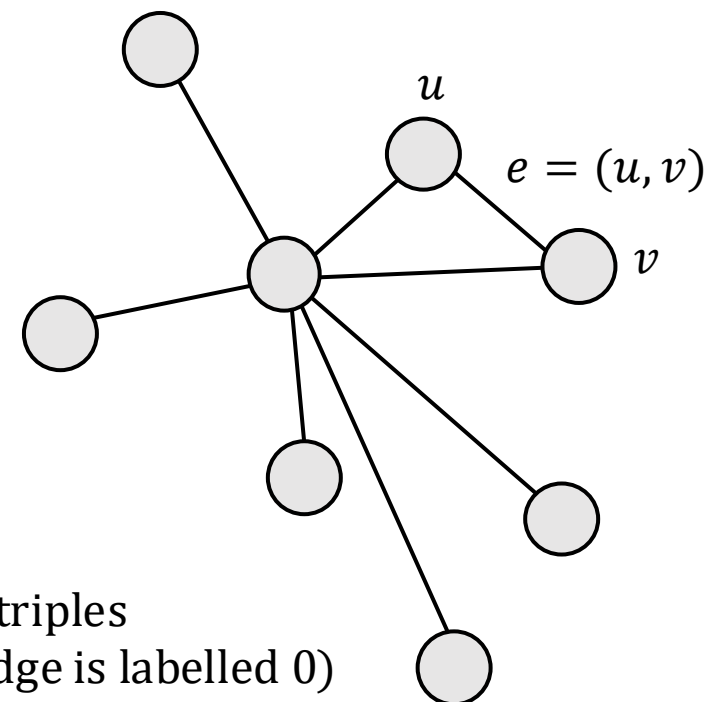
Adjacency list

$$E = \begin{bmatrix} [1,2] \\ [2] \\ [1] \end{bmatrix}$$

Set of triples

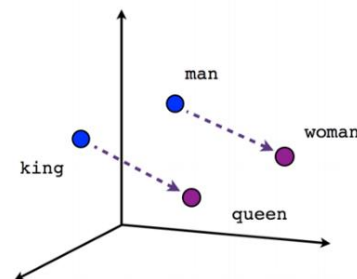
(the edge is labelled 0)

$$\{(0,0,2), (0,0,1), (1,0,2), (2,0,1)\}$$

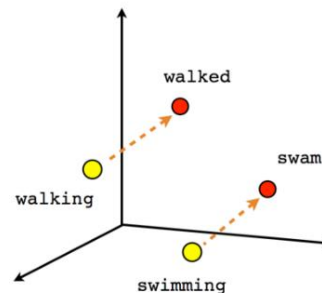


The ubiquity of matrices

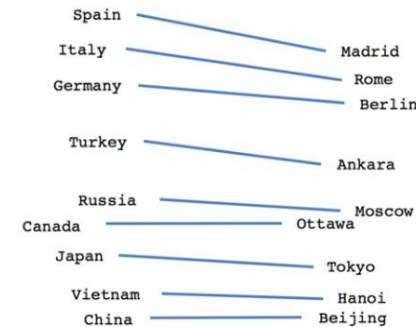
- Despite the different nature of text, images and speech data, matrices and vectors are used to represent all
- **Example:** Word embeddings (words represented as vectors)



Male-Female



Verb tense



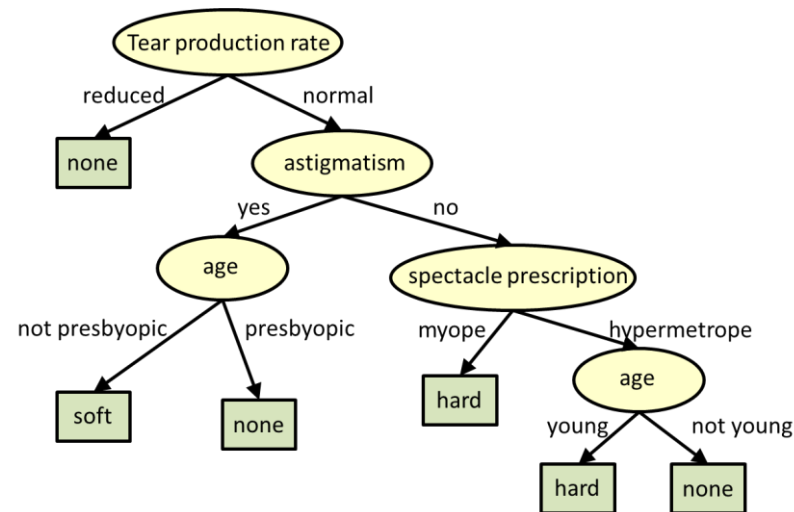
Country-Capital

Why matrices?

- Almost all ML builds heavily on linear algebra
 - **Linear regression:** $y = \beta^\top x + c$
 - **Deep neural networks:** $h_{l+1} = \sigma(W_{l+1}h_l + b_l)$
 - **Linear programming:**
$$\begin{array}{ll} \min & Ax + b \\ x & \\ \text{subject to} & Bx + c \leq 0 \end{array}$$
- Matrix/vector multiplications are fast, well-understood

Exceptions

- The most common exception, which does not (have to) build on matrix/vector operations is **decision trees**



Classifying type of contact lens needed

Numpy arrays and Tensors

```
>>> import numpy as np

>>> np.array([[1, 2], [3, 4]])
array([[1, 2],
       [3, 4]])
```



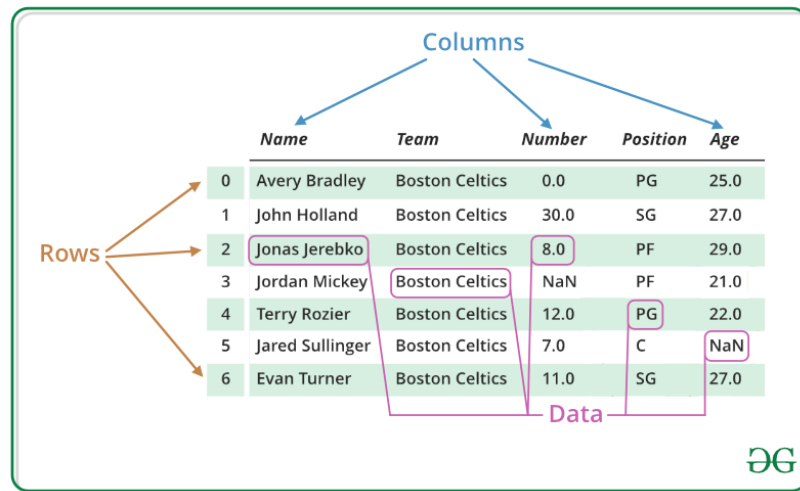
```
>>> import torch

>>> torch.tensor([[1., -1.],
                  [1., -1.]])
tensor([[ 1.0000, -1.0000],
        [ 1.0000, -1.0000]])
```



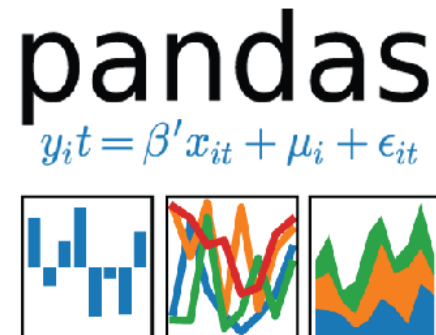
Data frames

- A **drawback** of pure matrix/tensor representations is that columns and rows and columns are “anonymous”
- **Data frames** add indices and names, much like a spreadsheet



The diagram shows a data frame with 7 rows and 5 columns. The columns are labeled Name, Team, Number, Position, and Age. The rows are indexed 0 to 6. A label 'Rows' points to the row indices. A label 'Columns' points to the column headers. A label 'Data' points to the data cells. The data is as follows:

	Name	Team	Number	Position	Age
0	Avery Bradley	Boston Celtics	0.0	PG	25.0
1	John Holland	Boston Celtics	30.0	SG	27.0
2	Jonas Jerebko	Boston Celtics	8.0	PF	29.0
3	Jordan Mickey	Boston Celtics	NaN	PF	21.0
4	Terry Rozier	Boston Celtics	12.0	PG	22.0
5	Jared Sullinger	Boston Celtics	7.0	C	NaN
6	Evan Turner	Boston Celtics	11.0	SG	27.0



	BandName	WavelengthMax	WavelengthMin
0	CoastalAerosol	450	430
1	Blue	510	450
2	Green	590	530
3	Red	670	640
4	NearInfrared	880	850
5	ShortWaveInfrared_1	1650	1570
6	ShortWaveInfrared_2	2290	2110
7	Cirrus	1380	1360

Handling different kinds of data

- Machine learning is still at a point where each data type is (best) handled by *slightly* different methods
- Although, they are merging! Largely thanks to matrix representation

Example: Multimodal LLM

Ours

a **man** sitting on a **boat** in the
a beach next to a middle of a
bird. body of **water**.



a black and
white photo of
a **clock tower**
in a city.



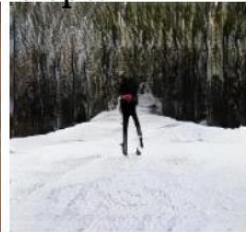
a **man** and a
woman
holding a
snowboard.



two slices of
pizza on a
wooden cutting
board.



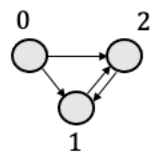
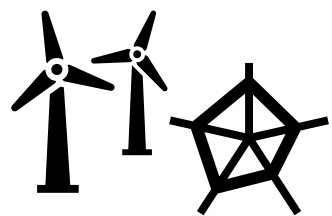
a **person** riding
skis down a
snow covered
slope.



a group of
people flying a
kite in a field.



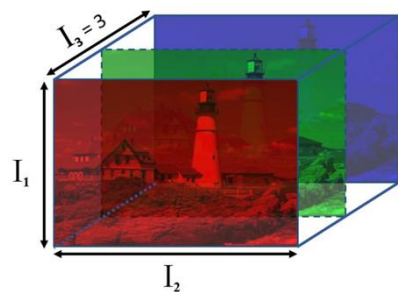
When can matrix representation go wrong



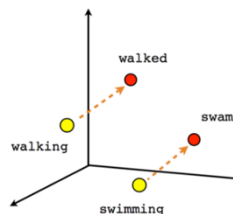
Adjacency matrix

$$G = \begin{bmatrix} 0 & 1 & 1 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Very
sparse



Extracted
features do not
capture the
essential
characteristics
of the images



Loss of context
and challenges on
high
dimensionality

-2	1	3	4
3	4	5	12
2	5	1	2
1	1	2	4

$$\rightarrow \theta \rightarrow Y$$

Data is not stored in matrices

- Most data is not stored with ML in mind
- **On the right:** report from investigation of an error in a street manhole in NYC*
- Even more structured data (e.g., SQL) is not always readily turned into a matrix
 - **Example:** The reason that the data exists had nothing to do with ML

```

ticket: ME000103052
lines: 31
Remarks: C.I.B POWERS REPORTS CONTRACTR WORKING I
N SB#158622 F/O 135
W.4 ST HAS 7W COPPERED AT W/DUCT -----JM
01/12/00 13:20 MDEO'HARA DISPATCHED BY 44729
01/12/00 14:00 MDEO'HARA ARRIVED BY 58101
01/12/00 14:25 OHARA REPORTS IN SB-158622 F/O 135 W. 4TH
ST HAVE 7W MAIN IN TROBLE GOING OUT THE WEST WALL....
UNABLE TO TELL IF IT GOES WEST OR SOUTH IN THE CROSSING
AS PER THE M&S PLATE... FLUSH REQUIRED... ORDERED... PM
01/12/00 17:16 OHARA REPORTS FLUSH TRUCK STILL WORKING
IN THE TROUBLE HOLE..... PM
01/12/00 20:10 OHARA REPORTS C/F/R 3-4/0DC,4-4/0AC,4"43'
FROM SB-158622 F/O 135 W.4TH ST TO SB-3627 F/O 136 W.4TH ST.
PARKING IS NORTH SIDE NO STANDING MON TO FRI 8AM TO 6PM
SOUTH SIDE METER PARKING 8AM TO 9AM MON TO FRI
-----
IN SB-3627 F/O 136 W.4TH ST CLEARED B/O'S. NEEDS C&R
IN SB-3622 F/O 135 W.4TH ST TIEING NUTRAL'S IN.....MB
01/12/00 20:40 OHARA REPORTS IN SB-158622 F/O 135 W.4TH ST
HE CLAERED ALL B/O'S AND RETIED ALL NUTRALS.....MB
01/12/00 20:40 MDEO'HARA COMPLETE BY 40075
***** E-00-12017-M *****
01/13/00 00:00 REFERRED TO: M.EC.UGBT ME1353 BY 07525
DOCS JOB: 019552984 TASK: 019552988 AWU: 019552989
01/13/00 02:45 ORIGINAL LAYOUT # E00-12017-M WAS ENTERED
IN ERROR THE CORRECT LAYOUT SHOULD READ AS FOLLOWED
E00-12018-M & HAS BEEN CORECTED ON THE ORIGINAL REFERAL
TICKET.....WNB
01/13/00 01:51 REFERRED TO: M.EC.CABBT EDSACB FY1 BY 07525
01/13/00 01:51 REFERRED TO: M.EC.UGBT EDSACB FY1 BY 07525

```

* Slide from Cynthia Rudin: https://www.youtube.com/watch?v=1hAiK_ZYl3I

From humans to matrices

- Data based on human note taking is notoriously difficult to turn into matrices/tables
- Humans annotate the same thing differently and leave things out if they deem it unnecessary.
 — What if what they left out is a column in our representation?

```

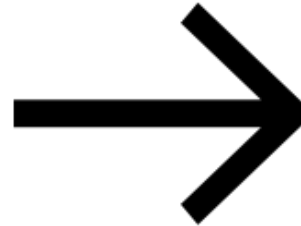
ticket: ME000103052
lines: 31
Remarks: C.I.B POWERS REPORTS CONTRACTR WORKING I
N SB#158622 F/O 135
W.4 ST HAS 7W COPPERED AT W/DUCT -----JM
01/12/00 13:20 MDEO'HARA DISPATCHED BY 44729
01/12/00 14:00 MDEO'HARA ARRIVED BY 58101
01/12/00 14:25 OHARA REPORTS IN SB-158622 F/O 135 W. 4TH
ST HAVE 7W MAIN IN TROBLE GOING OUT THE WEST WALL....
UNABLE TO TELL IF IT GOES WEST OR SOUTH IN THE CROSSING
AS PER THE M&S PLATE... FLUSH REQUIRED... ORDERED... PM
01/12/00 17:16 OHARA REPORTS FLUSH TRUCK STILL WORKING
IN THE TROUBLE HOLE..... PM
01/12/00 20:10 OHARA REPORTS C/F/R 3-4/0DC,4-4/0AC,4"43'
FROM SB-158622 F/O 135 W.4TH ST TO SB-3627 F/O 136 W.4TH ST.
PARKING IS NORTH SIDE NO STANDING MON TO FRI 8AM TO 6PM
SOUTH SIDE METER PARKING 8AM TO 9AM MON TO FRI
-----
IN SB-3627 F/O 136 W.4TH ST CLEARED B/O'S. NEEDS C&R
IN SB-3622 F/O 135 W.4TH ST TIEING NUTRAL'S IN.....MB
01/12/00 20:40 OHARA REPORTS IN SB-158622 F/O 135 W.4TH ST
HE CLAERED ALL B/O'S AND RETIED ALL NUTRALS.....MB
01/12/00 20:40 MDEO'HARA COMPLETE BY 40075
***** E-00-12017-M *****
01/13/00 00:00 REFERRED TO: M.EC.UGBT ME1353 BY 07525
DOCS JOB: 019552984 TASK: 019552988 AWU: 019552989
01/13/00 02:45 ORIGINAL LAYOUT # E00-12017-M WAS ENTERED
IN ERROR THE CORRECT LAYOUT SHOULD READ AS FOLLOWED
E00-12018-M & HAS BEEN CORECTED ON THE ORIGINAL REFERAL
TICKET.....WNB
01/13/00 01:51 REFERRED TO: M.EC.CABBT EDSACB FY1 BY 07525
01/13/00 01:51 REFERRED TO: M.EC.UGBT EDSACB FY1 BY 07525

```


Changes with the emergence of LLMs

ticket: ME000103052
 lines: 31
 Remarks: C.I.B POWERS REPORTS CONTRACTR WORKING I
 N SB#158622 F/O 135
 W.4 ST HAS 7W COPPERED AT W/DUCT -----JM
 01/12/00 13:20 MDEO'HARA DISPATCHED BY 44729
 01/12/00 14:00 MDEO'HARA ARRIVED BY 58101
 01/12/00 14:25 OHARA REPORTS IN SB-158622 F/O 135 W. 4TH
 ST HAVE 7W MAIN IN TROBLE GOING OUT THE WEST WALL.....
 UNABLE TO TELL IF IT GOES WEST OR SOUTH IN THE CROSSING
 AS PER THE M&S PLATE... FLUSH REQUIRED... ORDERED... PM
 01/12/00 17:16 OHARA REPORTS FLUSH TRUCK STILL WORKING
 IN THE TROUBLE HOLE..... PM
 01/12/00 20:10 OHARA REPORTS C/F/R 3-4/0DC,4-4/0AC,4"43'
 FROM SB-158622 F/O 135 W.4TH ST TO SB-3627 F/O 136 W.4TH ST.
 PARKING IS NORTH SIDE NO STANDING MON TO FRI 8AM TO 6PM
 SOUTH SIDE METER PARKING 8AM TO 9AM MON TO FRI

 IN SB-3627 F/O 136 W.4TH ST CLEARED B/O'S. NEEDS C&R
 IN SB-3622 F/O 135 W.4TH ST TIEING NUTRAL'S IN.....MB
 01/12/00 20:40 OHARA REPORTS IN SB-158622 F/O 135 W.4TH ST
 HE CLAERED ALL B/O'S AND RETIED ALL NUTRALS.....MB
 01/12/00 20:40 MDEO'HARA COMPLETE BY 40075
 ***** E-00-12017-M *****
 01/13/00 00:00 REFERRED TO: M.EC.UGBT ME1353 BY 07525
 DOCS JOB: 019552984 TASK: 019552988 AWU: 019552989
 01/13/00 02:45 ORIGINAL LAYOUT # E00-12017-M WAS ENTERED
 IN ERROR THE CORRECT LAYOUT SHOULD READ AS FOLLOWED
 E00-12018-M & HAS BEEN CORECTED ON THE ORIGINAL REFERAL
 TICKET.....WNB
 01/13/00 01:51 REFERRED TO: M.EC.CABBT EDSACB FYI BY 07525
 01/13/00 01:51 REFERRED TO: M.EC.UGBT EDSACB FYI BY 07525



Missing values

Example

- Say that we want to predict **cardiovascular risk** for patients who visit their general practitioner
- We look at electronic health record (EHR) data containing
 - X = Age, sex, weight, cholesterol
 - Y = Cardiovascular disease

Missing values

- When teaching/learning machine learning algorithms, inputs are often assumed to be fully observed

		$X =$				$Y =$	
		Age	Sex	Wght	Chol	Y	
Observations	Richard	30	M	85	4	-1	
	Kim	54	F	60	5	1	
	Anna	26	F	56	2	-1	
	Ivan	60	M	90	4	1	
		Features				Labels	

Missing values

- In reality, it is common for some feature values to be missing
 - **Example:** The doctor may consider cholesterol unnecessary to measure
- Observed values $\tilde{X}$ are a subset of the full features, X

		$\tilde{X} =$				$Y =$	
Observations		Age	Sex	Wght	Chol		Y
	Richard	30	M	85	?		-1
	Kim	54	F	?	5		1
	Anna	26	F	56	2		-1
	Ivan	60	M	90	?		1
		Features				Labels	

Question

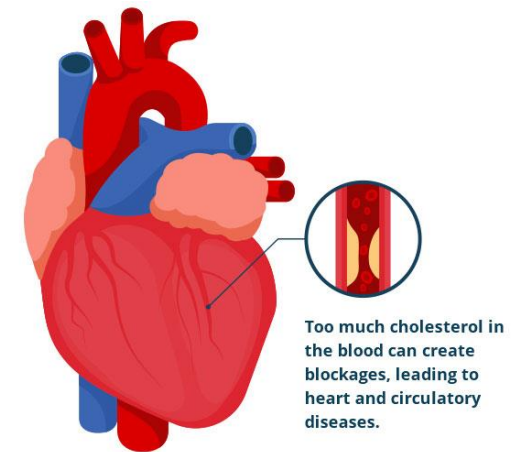
Suppose we want to **learn** to predict Y from **tabular** features X

What do we do with **missing values** in X ?

Take ~ 3 min and come up with as many solutions as you can

Example: Informative missingness

- For patients with no measurement of cholesterol, the GP probably saw no need to measure it
- Perhaps, at earlier consultations the cardiovascular risk was considered low
- Patients likely have a more favourable prognosis if the measurement is missing than if the measurement has been taken, **irrespective of the measured value.**



Missing values

- We can represent missing values by a missingness mask M

$\tilde{X} =$

	Age	Sex	Wght	Chol
Observations	30	M	85	nan
	54	F	nan	5
	26	F	56	2
	60	M	90	nan
	Features			

$$\tilde{X}_{ij} = \begin{cases} X_{ij} & \text{if } M_{ij} = 0 \\ \text{nan} & \text{otherwise} \end{cases}$$

$M =$

Age	Sex	Wght	Chol
0	0	0	1
0	0	1	0
0	0	0	0
0	0	0	1
Missingness indicators			

Two common solutions

1. Imputation

Attempt to impute missing values

1. Reconstruct X from $\tilde{X}$
2. Predict Y from reconstruction $\hat{X}$

Any method that works for X works for reconstruction $\hat{X}$

2. Informative missingness

Make use of missingness itself

1. Predict Y using both $\tilde{X}$ and M

Two common approaches:

- Method sensitive to M (e.g., XGBoost)
- Simple imputation + missing indicators

2. **Model** development

Outline: Model development

Model development

1. Reusable models
2. Differentiable models
3. Optimization libraries

Example: Building a recommender system

- Recall the assignment from last week
- We are tasked with building a **recommender system** for movies
- How should we **implement** this system?

Database of movie reviews

	User	The Net	Happily N'Ever After	Tomorrowland	American Hero	Das Boot	Fir
0	Vincent	0.0	0.0	0.0	0.0	0.0	0.0
1	Edgar	0.0	0.0	0.0	0.0	0.0	0.0
2	Addilyn	0.0	0.0	0.0	0.0	0.0	0.0
3	Marlee	0.0	0.0	0.0	0.0	0.0	0.0
4	Javier	0.0	0.0	0.0	0.0	0.0	0.0
5	Marcus	0.0	0.0	0.0	0.0	0.0	0.0
6	Mary	0.0	0.0	0.0	0.0	0.0	0.0
7	Rosalie	0.0	0.0	0.0	0.0	0.0	0.0
8	Giovanni	0.0	0.0	0.0	0.0	0.0	0.0
9	Kennedi	0.0	0.0	0.0	0.0	0.0	0.0
10	Alana	0.0	2.0	0.0	0.0	0.0	0.0
11	Silas	0.0	0.0	0.0	0.0	0.0	0.0
12	Kaitlyn	0.0	1.0	0.0	0.0	0.0	0.0
13	Michelle	0.0	0.0	0.0	0.0	5.0	0.0
14	Ophelia	0.0	0.0	0.0	0.0	0.0	0.0
15	Nevaeh	0.0	0.0	0.0	0.0	0.0	0.0
16	Mckenna	0.0	0.0	0.0	0.0	0.0	0.0
17	Emely	0.0	0.0	0.0	0.0	5.0	0.0
18	Lucas	0.0	0.0	0.0	0.0	0.0	0.0

Beginner mistake 1: “One long script”

- One of the **common pitfalls** in machine learning (and other) programming is the **one-long-script** tendency
- Data processing, training and testing written **one after another** in a single imperative script file
- Many tutorials follow this format! How can we improve?

Fit, predict, score

- A big development in ML is the **fit, predict, score** pattern
- Standardizes common ML **workflow**
 - `fit(x, y)`: Train model to e.g., predict y from x
 - `predict(x)`: Predict y from x
 - `score(x, y)`: Evaluate model on data x, y
- This is **part of your assignment** this week!

Overview

- Fit, predict, score fits neatly into **object-oriented programs**

```
class Classifier():  
    def fit(x, y):  
        ...  
  
    def predict(x):  
        ...  
  
    def score(x, y):  
        ...
```

```
C = Classifier()  
C.fit(X_train, Y_train)  
C.predict(x_new)
```

fit(x, y)

- The function **fit(x, y)** is responsible for training and storing model parameters that maximize the objective
- For example, finding the optimal coefficients in OLS or k-means.
- The arguments x, y **may vary** depending on application
- In **unsupervised learning**, there is no y, but parameters to fit!

predict(x)

- The function **predict(x)** should take a (new) data point x and predict the outcome (e.g., label/cluster) for it
- Separating predict from fit reduces the risk of the common mistake of making **one-use code** that relies on the training data
- However, sometimes, like in k-nearest neighbor classifier, $\text{predict}(x)$ will **explicitly depend** on the training data

score(x, y)

- The function **score(x, y)** assigns a score to the prediction made for x in comparison to the label y
- Separating score from $\text{fit}(x, y)$ makes it easy to evaluate both training and test errors and **reduces the risk of label leakage**
- $\text{score}(x, y)$ can also be used for **hyperparameter selection**, for example using cross-validation

Scikit-learn (sklearn)

- Contains standard learning algorithms, toy datasets and more
- Centered around “**estimators**”
 - Implementations of machine learning models for classification, regression, density estimation, etc.
- One of the forerunners of “fit, predict, score”
- Also, **transformations, metrics, parameter search**



Data preprocessing

- All AI/ML algorithms are **sensitive to the representation** of the input data—often more than we would like
- **Trivial example:** The coefficients of OLS (linear regression) depend on the **scale of the covariates**

$$y = \beta^\top(ax) + c = (a\beta)^\top x + c$$

- 1m = 100cm, but how do we compare “1m” to “December”?

Data preprocessing

- A common step in AI/ML is to “**standardize**” features, i.e., to make them have mean 0 and standard deviation 1

$$X' \leftarrow \frac{X - \mu(X)}{\sigma(X)}$$

- Strictly speaking, μ and σ are **parameters**. They are functions of the training data
- For **a single test point**, there is no σ (or it is 0). We should use the values from the training data!

Transformations

- Scikit-learn implements data preprocessing in **transformations**
- They have similar syntax as estimators, but **fit(x)**, **transform(x)**
- Can be used for standardization, discretization, *missing-value imputation*, feature generation, etc.

Value of design patterns

- You don't **need** to use “fit, predict, score” always—sometimes it doesn't fit the task
- Reinforcement learning, for example, **does not fit** well into this paradigm. It relies on “roll-outs” rather than single predictions
- However, **adapting standards** for inputs and outputs reduces the risk of bugs and makes widespread use easier!

3. **Model** evaluation

Recap Train/val/test split

Full dataset

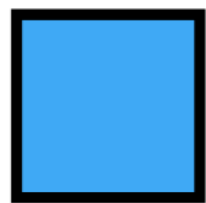


```
sklearn.model_selection.train_test_split(*arrays, test_size=None,  
train_size=None, random_state=None, shuffle=True, stratify=None)
```



Cross validation

$n = 8$



Test



Train

Model 1



Evaluation metrics

- Regression: RMSE, R^2 , ...
- Classification: Accuracy
- ...
- Confusion matrix

		Predicted class	
		Positive	Negative
Actual class	Positive	TP	FN
	Negative	FP	TN

sklearn.metrics

Score functions, performance metrics, pairwise metrics and distance computations.

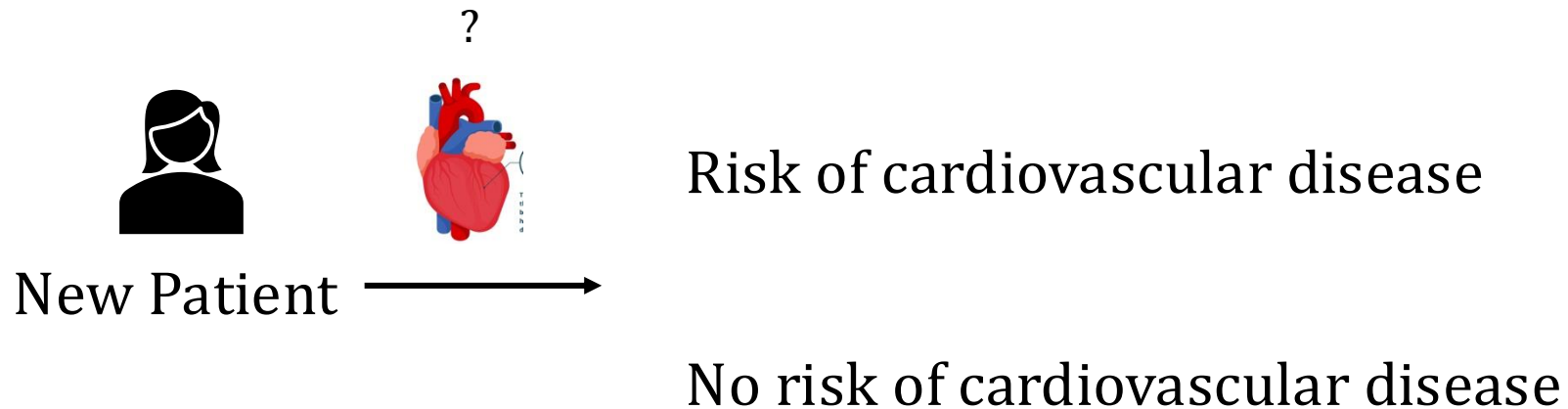


```
sklearn.metrics.confusion_matrix(y_true, y_pred, *,
labels=None, sample_weight=None, normalize=None)
```

Differentiable systems

Learning as Optimization

- **Each time** we fit a machine learning algorithm on a training dataset, we are **solving an optimization** problem.



Recap: Empirical risk minimization (ERM)

- Finding parameters θ that **minimizes loss L on observed data**

$$\min_{\theta} \frac{1}{n} \sum_{i=1}^n L(f_{\theta}(x_i), y_i)$$

- ERM is by far the most common AI/ML strategy today
- **How do find the optimal θ ?**

Differentiable systems

- Deep Learning and many other AI tools are based on **ERM** in **differentiable systems**
- Systems where the objective function is differentiable in the parameters θ . **Recall:** the ubiquity of matrix multiplication
- This means that learning using **gradient descent** is possible

Empirical risk minimization

- Consider the **linear model**, $f(x) = \theta^\top x$, of a 1D label y
- We can measure its error using the **mean squared error**

$$\hat{R}(\theta) = \frac{1}{n} \sum_{i=1}^n (\theta^\top x_i - y_i)^2$$

- How do we find the parameters θ^* which minimize $\hat{R}(\theta)$?

Recap: Gradient descent (GD)

- We can use gradient descent to find a (local) minimum!

$$\hat{R}(\theta) = \frac{1}{n} \sum_{i=1}^n (\theta^\top x_i - y_i)^2$$

- Gradient descent says to move in the direction of $-\nabla(\hat{R})$

$$\theta_k \leftarrow \theta_{k-1} - \eta \cdot \nabla_{\theta}(\hat{R})$$

- with step-length η

Gradient/derivative of $\hat{R}$ w.r.t. θ

Here, $\nabla_{\theta}(\hat{R}) = \frac{2}{n} \sum_{i=1}^n (\theta^\top x_i - y_i) x_i$

Automating gradient descent

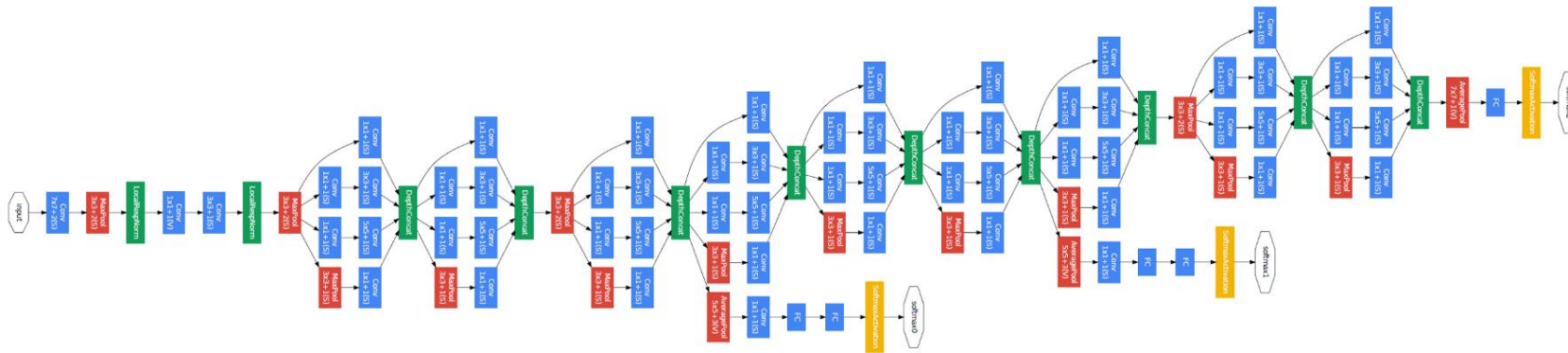
- GD has only **one non-trivial operation**—computing the gradient

$$\theta_k \leftarrow \theta_{k-1} - \eta \cdot \nabla_{\theta}(\hat{R}) \qquad \nabla_{\theta}(\hat{R}) = ?$$

- As long as $\hat{R}$ is a composition of differentiable functions of θ , differentiation is easy—exploited in modern ML!

Deep learning

- A famous example of this is **back-propagation** in deep learning!



The “Inception” image classification model

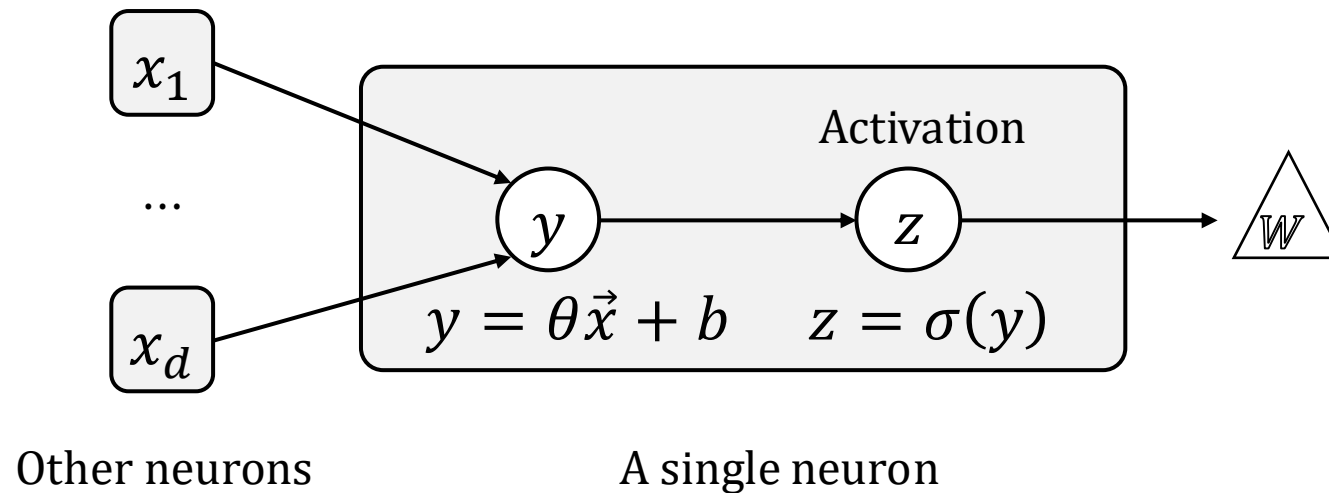
- “Back-propagates” classification error to model parameters

Symbolic differentiation

- If each component of the graph is **simple enough**, we can express its gradient analytically, as a function of its arguments
- For example, $\nabla_{\theta}(\theta^{\top}x) = x = [x_1, \dots, x_d]^{\top}$
- If we know this for all nodes in our computation graph, the chain rule **gives us the gradient** of all nodes w.r.t. all nodes!

Example: Units in deep learning

- Typically, units (neurons) in artificial neural networks are compositions of **linear functions** and non-linear **activations**



- Activations with “simple” gradients preferred, e.g., $\sigma(x) = \frac{1}{1+e^{-x}}$

Symbolic differentiation at scale

- This simple idea underlies huge frameworks for deep learning



- ... the trick is to make it **easy to build and work with** the computation graphs—and to make gradient computation **fast**!
- You'll see more of this in **Applied Machine Learning**!

Question

Where do you think **gradient descent** is not a good choice?

Take ~3 min and come up with as many solutions as you can

A brief disclaimer

- Except in simple problems (e.g., convex in parameters), gradient descent gives **few guarantees of optimality**
- The **jury is still mostly out** on why deep learning works so well

(More general) **Optimization** tools

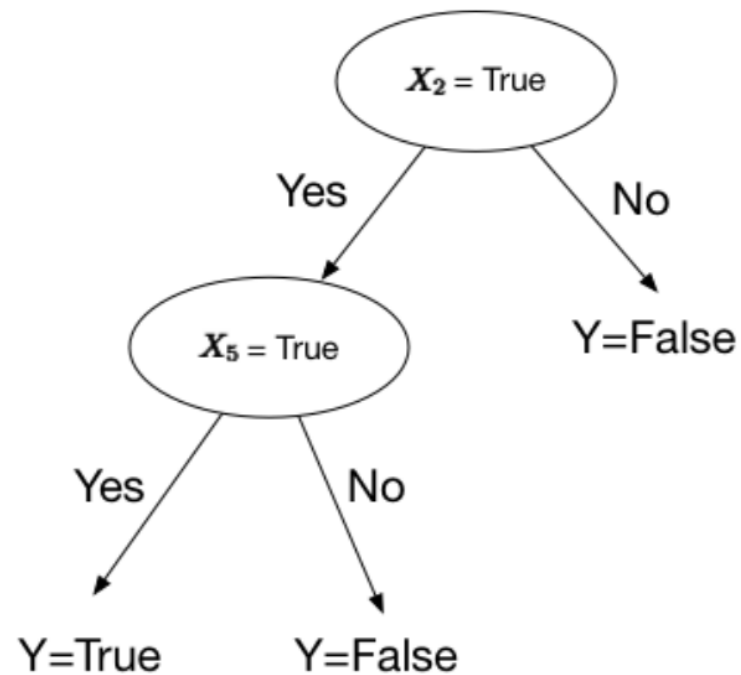
Constraints, Example: Fair ML

- In "fair" machine learning, one goal is for different groups $a \in \{0,1\}$ (e.g., men and women) to have **equal opportunity**
- This may be defined as the **constraint** of having **equal probability of predicted** success given true success, $\gamma_a(\hat{Y}_\theta)$

$$\begin{array}{ll}
 \underset{\theta}{\text{minimize}} & R(\hat{Y}_\theta) \quad \text{Risk} \\
 \text{subject to} & \gamma_0(\hat{Y}_\theta) = \gamma_1(\hat{Y}_\theta) \quad \text{——— Constraint!} \\
 & \text{Equal true positive rate}
 \end{array}$$

Non-differentiable, Example: Optimal decision trees

- Decision trees are **almost always** trained using greedy algorithms
- Tree structure is **discrete**, its parameters non-differentiable



Support rules $\mathcal{S}$

$$\hat{O} = S.1 \wedge (B.1 \vee B.2 \vee B.3 \vee B.4)$$

Rule S.1:

History:	and	Surgical procedure:
$\neg$ Injury of face and neck		$\neg$ Endocrine system
and $\neg$ Unspecified septicemia		and $\neg$ Mediastinum (thoracic cavity)
and $\neg$ Other injury of chest wall		and $\neg$ Auditory system
and $\neg$ Acute respiratory failure		and Age $\in [0, 64]$
and $\neg$ Altered mental status		

Propensity overlap rules $\mathcal{B}$

Rule B.1:

Surgical procedure:
Musculoskeletal

or Rule B.2:

Age > 44
and Male
and Surgical procedure:
Cardiovascular
and $\neg$ Urinary system (e.g, bladder)
and $\neg$ Male genital system

or Rule B.3:

Surgical procedure:
Nervous (e.g., epidural)
and $\neg$ Maternity (e.g., C-section)
and $\neg$ Female genital system

or Rule B.4:

Age > 23
and Surgical procedure:
$\neg$ Maternity
and History:
Thoracic or lumbosacral neuritis or radiculitis

Optimization in general

- Optimization is a much broader topic than gradient descent!

$\underset{x \in \mathcal{X}}{\text{minimize}} \quad f(x)$	Objective
$\text{subject to} \quad g(x) \leq 0$	Inequality constraint
$h(x) = 0$	Equality constraint

- What if $\mathcal{X}$ is a discrete set? What if $g(x)$ is a quadratic function?
What if f is not differentiable?

Optimization tools

- Luckily, **there are tools** also for constrained and non-differentiable problems
- **Famous examples** are CVX (cvxpy) and Gurobi (gurobipy)
- These tools provide a syntax for **specifying optimization problems** and solvers for solving them



GUROBI
OPTIMIZATION



CVX
RESEARCH

See you next time!